GitFit

Group 17

Members: Aluse, Gurjot, Harman, Jagraj

Repo, Website, Video Demo

Analysis of Project Success:

Goal: Allow gym goers(new to fitness industry or already going) to be able to conveniently track their fitness progress and be able to generate or create workouts no matter how much time they have.

Result: Overall, the project succeeded in meeting our goals. Users who are new can generate a workout based on the weather or create their own custom workout, with our workout builder. This allows the user to create a workout based on their preferences and time constraints. Additionally, the step tracking feature allows the user to monitor their day-to-day progress by providing a visual representation of their daily steps walked. By addressing the user's needs for getting started in the gym and progress tracking. The project solved the main goal we had.

User testing (In-class): When we did in-class testing we received amazing user feedback. When first creating a recommended workout, the recommended workouts were missing proper formatting. Thanks to user testing we were able to fix this issue. Another user said that they found it easy to add steps and navigate they GitFit website. And additionally, getting a recommended workout was also easy for them. When peer-testing another group we saw when the website first loaded up they had modals that guided the users on how to start using the website. As a group we talked about implementing this idea, and we all agreed that this will be an amazing change. Thanks to the peer-testing we were able to make amazing progress that we would not have been able to make ourselves.

Analysis of Project SDLC:

Chosen SDLC: The SDLC method was a hybrid of a few popular SDLC methods that we were taught in class. The main inspiration was Agile. We had weekly meetings (on Sundays or Saturdays). We chose not to do daily stand-ups. As a group we had a lot of conflicting schedules and in the end were unable to find a time that worked for all of us. Instead, what we did is we would write a small report every two days that answered the questions: What did you do? What challenges did you face? Do you require any assistance? What will you be until the next reporting period? This was the method that worked for all of us and it did amazing.

Changes made to SDLC: No changes were made to the SDLC method over the course of the semesters.

API Feature Description

Weather API(OpenWeatherMap API):

Features

Allow the user to plan their workout based on the weather

Added a recommendation button to workouts page. When clicked, the site checks the weather and selects from predefined workout categories based on the temperature. These categories are: "Cold"(-100°C to 5°C) for arms, "Average" (6°C to 15°C) for legs, "Warm" for abs (16°C to 25°C), and "Hot" for cardio (26°C to 100°C). These categories were chosen because it's easier to work out arms and legs in colder weather, while abs and cardio are more suited for warmer temperatures. Once a category is selected, the site uses the Wger API to suggest appropriate exercises. A failsafe was also implemented to ensure that outdoor workouts aren't recommended during unsafe weather conditions such as rain, snow, thunderstorms, or drizzle, instead advising the user to do indoor workouts.

Clothing and Gear Suggestion

Like the workout recommendation system, the site also provides clothing and gear suggestions based on the weather. When the user interacts with the recommendation button, the API checks the current weather conditions for categories clear, rain, snow, clouds, drizzle, haze, smoke, and thunderstorms. Based on the detected weather condition, a set of predefined clothing options for each category is selected, and a random clothing element from the set is then presented to the user.

Weather page

The weather page was created to provide users with more detailed information about their city's weather. The page displays the current temperature and specific weather conditions, such as moderate rain. Additionally, users can search for a specific city, and the system will fetch and display the temperature and weather condition for that city.

The city selected is then used across all weather-related features on the site, including workout recommendations, clothing suggestions, and weather info.

Weather info

A weather info display was added to the top-right of the navbar to allow users to quickly view the temperature and weather condition for their registered city. This display provides confirmation that the correct city is selected, and the information is fetched based on the city entered on the weather page. This feature ensures that users don't need to navigate to the weather page repeatedly to check their city's current weather.

Changes

Motivational messages

Originally, the plan was to offer motivational messages tailored to the current weather conditions, but this feature was removed. The intention was to help motivate users and at the same time provide weather info, but it was determined that it could become distracting and would not align with the GitFit theme of simplicity and ease of use. Instead, a constant weather info display on the navbar and a dedicated weather page for more detailed information were introduced, providing users with weather updates in a less intrusive manner.

Weather page

The weather page was designed to allow users to input their city, which would then be used across all API features to fetch the corresponding weather information. Initially, the plan was to incorporate this feature into the profile page, but due to its importance, it was decided to create a separate weather page for better clarity and user experience. This page allows users to check and change their city settings easily, while also providing immediate feedback on whether the city was successfully registered on the site. By having a dedicated page for

weather, it reduces clutter on the profile page and ensures users can quickly access the weather information they need.

Weather info

This was added to let users quickly check the temperature and confirm their registered city without repeatedly visiting the weather page.

WGER API(Exercise API):

- Random Exercise Generator
    - Gives the user a set of recommended exercises based on the current weather conditions. The exercises are tailored to the outside temperature and provide a list of recommended clothes to wear for the workout.
- Exercise Tracker
    - Allows the user to create and customize a list of exercises that saves to website locally. The user can adjust the sets and repetitions of each exercise, delete exercises, or reset their current workout split back to default.
- Exercise Search
    - Interactive search bar, that allows the user to search through the WGER API exercises endpoint and discover new exercises.

Changes since Planning Stage:

- Nutrition Page:
    - Removed the nutrition page, as we believed that utilizing its features and creating customized meal plans would be too difficult and time constraining.
- Weather Page:
    - We decided to change the nutrition page into a weather page, utilizing a weather API. The weather page allows the user to enter the city they are currently in, and the website will then display the

current temperature and forecast and will recommend exercises along with the WGER API.

CI/CD pipeline:

We used GitHub actions to create a workflow that would automate the testing process. This automated testing is called everything there is a pull or push request made onto the main branch. Listed down below is detailed process of how it works:

- First: there is a checkout, this retrieves the latest version of the code.
- Second: On the GitHub runner node.js is installed
- Third: all dependencies from the package.json are installed to ensure they testing environment meets the project requirements.
- Fourth: All tests from the "tests" folder are run.

Testing Strategy:

Our projects testing strategy used both unit testing and integration testing. This ensures testing for all core features. To create these tests we used Jest and React Testing Library. Unit testing focused on logic like workout creation, weather-based recommendation, and app rendering. While the integration tests verified API fetching from the API was correct.

Tests

1. The first test is App.test.js and can be found in: code/src/App.test.js. This test checks if the GitFit dashboard renders correctly. This check is done by first rendering the page and checks if the heading "GitFit" shows up on the page

2. The next test is exerciseSearch.test.js and can be found in: code/src/__tests__/exerciseSearch.test.js. This test does a lot more than the previous one. This test searches for exercises using the alias: https://wger.de/api/v2/exercisealias/. There are three test cases for this test. Two of the tests are searches for a word “jump” and “squat” and returns at least one matching term. The last test is an input that does not exist and should return nothing, the input we used is “sadasd.” This test ensures that the exercise look up feature is working and returns valid results.

3. The next test is getWeather.test.js and can be found in: code/src/__tests__/clothingRecommendation.test.js. This test ensures that the “getWeather” function correctly fetches and formats the weather data. It uses “jest.fn()”, it mocks and simulates a successfully API response for a city, in the case of this test it is “New York.” This test ensures that the data is formatted properly and “getWeather” can handle API responses.
4. The next test is clothingRecommendation.test.js and can be found in: code/src/__tests__/clothingRecommendation.test.js. This test ensures that the “getClothingRecommmendation” functions gets and suggests proper clothing based on weather conditions. It has three test conditions, testing for normal weather conditions like “rain” or “sunny”, the second test, tests for abnormal weather conditions like “raining dogs,” the final test checks for case sensitivity like “RaIN” or “rAIN.”
5. Next is the getWorkouts.test.js and can be found in: code/src/__tests__/getWorkouts.test.js. This test creates a mock fetch request and makes sure that the function returns an array of exercises, and it contains at least one workout.
6. The next test is openWeatherAPI.test.js and is located in code/src/__tests__/openWeatherAPI.test.js. It tests an API call and gets weather data from a city. In the case of this test, it is Vancouver. It

verifies three things. A 200-response code, object data contains a main property, and main contains the temperature.

7. The final test is is wgerAPI.test.js and makes an API call and ensures that the call receives exercise data. It verifies it is a 200-response code, the response contains results, and the results contain at least one exercise.

Known Bugs:

Bugs	Severity
When searching for an exercise some times exercises in different languages will show up, specifically in the German language. How to reproduce: 1. Go to the workouts page 2. In the search Bar enter “f” 3. Some of the shown results are names for exercises in the German language	1(Minor)
When exercises are added some exercises are longer in terms of words. This slightly shifts the buttons. How to reproduce: Go to workouts page	1(Minor)

Add multiple exercises of different lengths, you can see button slightly shifted.	

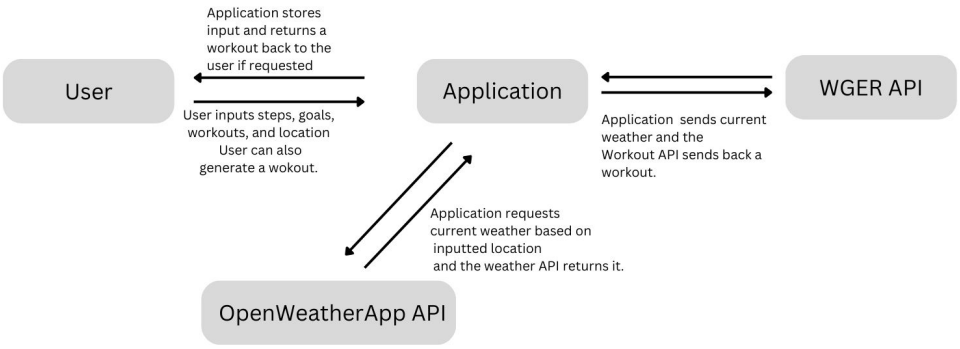
Future Work:

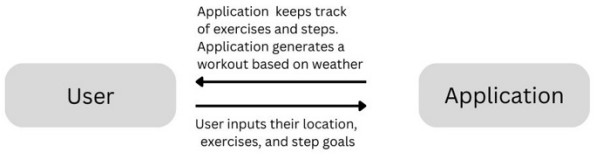
1. The recommendation to take more factors into consideration such as goals (weight loss, muscle gain, weak points, etc...). It could do this by recommending more cardio during weight loss or putting exercises for weak points first
2. A feature to “start” a workout. It will tell you your current exercise and sets left. The user will need to click a button to indicate they have finished that set. Additionally, it will keep track of rest times between each set.
3. Instead of manually inputting steps only, the application can connect to apps such as Apple's health app, to automatically track steps. The manual input can still be used if needed.
4. Reminders to get your steps in. If you are far from your step goal at around 8pm, it should send a notification.
5. A button or hover effect for each exercise that gives a description of it. It should display the muscle worked, potentially risky (since it is easy to hurt yourself on some exercises compared to others), what is it good for (strength, hypertrophy, athleticism) etc.

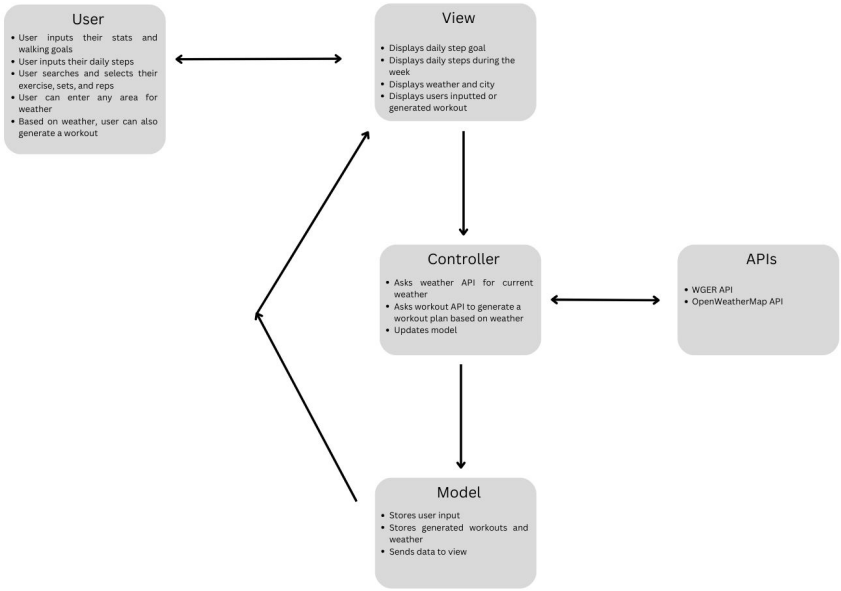
Lessons Learned and Project Takeaways:

1. How to work together as a team and collaborate on a project at the same time
2. How to learn on the go especially when there is limited time to learn
3. Knowing how to search for a specific issue
4. Knowing how to ask for help, especially when you really need it.

Updated MVC and DFD diagram:







Appendix:

Contributions:

Aluse:

- Created all the DFD (Data Flow Diagram) and MVC (Model-View-Controller) diagrams.
- Designed and styled the dashboard to match the Figma design.
- Completed the peer review for Group 20.
- Removed the placeholder search bar from the workouts page.
- Added the reps and sets feature to workouts.
- Created updated versions of the DFD and MVC diagrams.
- Wrote and added 3 JavaScript tests.
- Ensured that all usability heuristics were met.

Gurjot:

- Created the Workout page and implemented all required features to match the Figma design.
- Completed the peer review for Group 21.
- Fully implemented the WGER API to fetch exercise data.
- Built the Workout Builder feature:
- Allows users to search for exercises and add them to a table.
- Integrated localStorage to save the user's workout.
- Contributed to Milestone 2 by:
    - Writing the WGER API description.
    - Creating a list of future features to implement.

Harman:

- Added CSS styling across pages to enhance visual consistency.
- Completed the peer review for Group 22.
- Created the recommendation section on the Exercises page:
- Displays clothing suggestions based on the weather condition.
- Implemented a feature where searching a location on the Weather page updates the entire website's location context.
- Wrote documentation for all code and performed code cleanup.
- Contributed to Milestone 2 by:
- Writing the OpenWeatherMap API description for M2.
- Creating the project video for Milestone 2.

Jagraj:

- Set up the OpenWeatherMap API and added location filtering functionality.
- Completed the peer review for Group 18.
- Implemented localStorage for both steps tracking and the steps graph.
- Added a day-based tracking feature, storing steps based on the current day.
- Wrote and added 3 JavaScript tests.
- Updated the README file with setup and usage instructions.

- Deployed the website on vercal and created a custom domain name.
- Created and configured GitHub Actions workflows for CI/CD.

Change-Log(Since M1): No changes have been made since milestone 1.

Peer testing feedback form (survey questions):

User Tests:

Test 1: Add Steps

- Navigate to the dashboard page and in the text box field enter the number of steps you have walked today
- Try adding more steps and see your results on that graph and bar graph change

Test 2: Get and Outfit Recommendation based on your local weather

- Navigate to the Workouts page
- On the workouts page scroll to the bottom and click “Get New Recommendation”
- This should give you a workout based on your local weather
- Check if the clothing/attire is suitable for the current weather and temperature.

Test 3: Search your local weather

- Navigate to the weather page
- On their enter you local city
- Type in the City,Province,Counter
    - For example Surrey,BC,CA
- Then at the bottom there should be your areas local temperature

Feedback

Test 1:

1. How difficult was it to add steps (on a scale of 1-10, 1 being easy and 10 being hard)
2. In the process of adding steps did you find anything challenging?
3. Did you encounter any issues adding steps?

Test 2:

1. How hard was it to get a recommended clothing suggestion based on the weather?(on a scale of 1-10, 1 being easy and 10 being hard)
2. Did you encounter any issues in getting clothing recommendations?
3. How easy was it to navigate the page and get a clothing recommendation?

Test 3:

1. How hard was it to navigate to this page?
2. How difficult was it to find your city?
3. Did you encounter any errors or issues?

User Feedback Summary

Step Counter Interaction

- User interacted with the step counter and successfully added 36 steps.
- Additional steps were added with ease.
- Max limit of 10,000 steps: Display an error message when the limit is exceeded.
- Third-person usability: Users found it easy to add steps and appreciated the responsive interaction.

Recommendations Section

- There is a formatting error with commas in the recommendations list.
- The text currently says "Suits on workout page" — but the recommendation could logically appear on either the Weather or Workout page.
- Second-person phrasing noted; consider consistent tone (e.g., use third-person or neutral).

Weather Page

- Recommendation section shifts when navigating to the Weather page.
- Users suggested moving or clarifying the recommendation’s placement on this page.

Navigation and Search

- Users found the app easy to navigate.
- The city search feature was intuitive and simple to use.
- Suggestion: Add a dropdown bar to the city search for quicker selection.

General Usability

- The recommendation system was described as easy to understand and take action on.
- Overall, users experienced smooth interactions and found core features accessible.